React Native Interview Questions — Topic 1: Core Components (150 Q&A)

Section A — View (15)

1. Q: What is `View` in React Native?

A: It's the most fundamental building block — a container component similar to a `div` on the web, backed by a native `UIView` (iOS) or `ViewGroup` (Android). It supports Flexbox layout, styling, and touch handling.

2. Q: Does `View` scroll its content?

A: No. `View` does not scroll — content that overflows its bounds is clipped or overflows visually depending on the `overflow` style. Use `ScrollView` or `FlatList` for scrollable content.

3. Q: Can you put raw text directly inside a `View`?

A: No. Unlike HTML, RN throws an error ("Text strings must be rendered within a `<Text>` component") if you put a bare string as a child of `View`.

4. Q: What layout system does `View` use?

A: Flexbox, via Yoga (Facebook's cross-platform layout engine), with some CSS-inspired defaults changed (e.g. `flexDirection: 'column'` by default).

5. Q: Is `View` accessible/focusable by default?

A: Not by default. You need to set `accessible={true}` and related `accessibility*` props for it to be treated as a single accessible element by screen readers.

6. Q: How do you handle touches on a `View`?

A: `View` itself has no built-in press handling; you wrap it in `Pressable`, `TouchableOpacity`, or use the `onStartShouldSetResponder`/gesture responder system directly.

7. Q: What does `overflow: 'hidden'` do on a `View`?

A: It clips children that extend beyond the view's bounds, similar to CSS. Default is `visible` on iOS in some RN versions, but you should set it explicitly.

8. Q: Can `View` have a background image?

A: Not directly — `View` only supports `backgroundColor`. For background images you use `ImageBackground`, which wraps an `Image` and a `View` together.

9. Q: How does `View` handle `zIndex`?

A: `zIndex` works only among siblings within the same parent and only meaningfully affects rendering order when using absolute/relative positioning together with overlapping views.

10. Q: What's the difference between `View` and `SafeAreaView`?

A: `SafeAreaView` is a specialized `View` that automatically adds padding/insets to avoid notches, status bars, and home indicators; a plain `View` has no such awareness.

11. Q: Can a `View` be animated?

A: Yes, typically via `Animated.View` (from the `Animated` API) or `Reanimated`'s `Animated.View`, which interpolate style props like `opacity`, `transform`, etc.

12. Q: What native component does `View` map to on Android?

A: A `ReactViewGroup`, which extends Android's `ViewGroup`.

13. Q: How do nested `Views` affect layout performance?

A: Deeply nested view hierarchies increase layout calculation cost (Yoga has to traverse more nodes) and can slow down initial render and re-layout; flattening hierarchies helps performance.

14. Q: Does `View` support `pointerEvents`?

A: Yes — `pointerEvents` (`'auto'`, `'none'`, `'box-none'`, `'box-only'`) controls whether the view and/or its children can be touch targets.

15. Q: Can you give a `View` a border-radius per corner?

A: Yes, using `borderTopLeftRadius`, `borderTopRightRadius`, `borderBottomLeftRadius`, `borderBottomRightRadius` individually, in addition to the shorthand `borderRadius`.

Section B — Text (15)

16. Q: Why must all text be wrapped in `<Text>`?

A: Because RN's native text rendering (especially on iOS with `NSAttributedString`) needs to know the full styled text run ahead of time; there's no browser-style inline text layout engine, so text nodes must be explicit.

17. Q: Can `Text` components be nested?

A: Yes — nesting `Text` inside `Text` lets you apply different styles (like bold or colored spans) to parts of a sentence, and styles are inherited/merged from parent to child.

18. Q: Does `Text` support `onPress`?

A: Yes, `Text` has a built-in `onPress` prop, making it touchable without wrapping in a `Pressable`, useful for things like inline links.

19. Q: How do you truncate long text in RN?

A: Use `numberOfLines` combined with `ellipsizeMode` (`'head'`, `'middle'`, `'tail'`, `'clip'`) to truncate text and control where the ellipsis appears.

20. Q: What prop selects text for copying?

A: `selectable={true}` makes text user-selectable (mainly relevant on iOS/Android long-press-to-copy).

21. Q: How do you control letter spacing and line height?

A: Via the style props `letterSpacing` and `lineHeight`, applied like any other style on `Text`.

22. Q: Is `Text` styling inherited by nested `Text`?

A: Yes, style properties cascade down to nested `Text` children, unlike `View`, which does not have style inheritance for its children.

23. Q: What's `adjustsFontSizeToFit` for?

A: An iOS/Android prop that shrinks font size automatically so text fits within its container, often combined with `numberOfLines` and `minimumFontScale`.

24. Q: Can you disable font scaling from device accessibility settings?

A: Yes, with `allowFontScaling={false}`, though it's discouraged for accessibility — it prevents the text from respecting the user's system font size preference.

25. Q: What happens if you don't specify a font family?

A: RN falls back to the OS default system font (San Francisco on iOS, Roboto on Android), which differs per platform unless you load a custom font.

26. Q: How do you underline or strike through text?

A: With the `textDecorationLine` style prop: `'none'`, `'underline'`, `'line-through'`, or `'underline line-through'`.

27. Q: Does `Text` support `ellipsizeMode` without `numberOfLines`?

A: `ellipsizeMode` only takes effect when `numberOfLines` is also set; without a line limit there's nothing to ellipsize.

28. Q: How is text measured for layout in RN?

A: The native text rendering system (Yoga + platform text layout, e.g. `NSAttributedString` bounding rect or Android `StaticLayout`) measures actual glyph metrics, which is why measuring can differ slightly from CSS-based web tools.

29. Q: Can `Text` contain an `Image`?

A: Yes — nesting an `Image` inside `Text` is supported so it flows inline with the text, similar to an inline image in rich text.

30. Q: What's the performance concern with many nested `Text` spans?

A: Each additional nested `Text` requires extra native measurement/layout work; excessive nesting for rich text can slow rendering, so libraries often flatten styled spans internally.

Section C — Image (15)

31. Q: Why doesn't `Image` auto-size like an HTML `<img>`?

A: Because RN doesn't know image dimensions ahead of network fetch/layout the way a browser progressively does, so you must specify `width`/`height` (or use `resizeMode`/aspect ratio tricks) up front to avoid layout jumps.

32. Q: What are the two main ways to source an image?

A: `require('./local.png')` for bundled local assets (resolved and sized automatically at build time) and `{ uri: 'https://...' }` for remote images (dimensions unknown until loaded unless specified).

33. Q: What does `resizeMode="cover"` do?

A: It scales the image to fill the given dimensions while preserving aspect ratio, cropping any overflow — similar to CSS `background-size: cover`.

34. Q: What does `resizeMode="contain"` do?

A: It scales the image to fit entirely within the given box while preserving aspect ratio, potentially leaving empty space — like CSS `background-size: contain`.

35. Q: How do you show a placeholder while a remote image loads?

A: Use the `onLoadStart`/`onLoadEnd` callbacks with local state, or use `ImageBackground`/overlay a loader, or libraries like `expo-image`/`FastImage` that support built-in placeholders.

36. Q: What is `Image.getSize()` used for?

A: It asynchronously retrieves the width and height of a remote image before rendering, letting you calculate aspect-ratio-correct dimensions dynamically.

37. Q: How does RN handle image caching?

A: On iOS, `Image` uses the system `NSURLCache` by default; caching behavior on Android is more limited, which is why many teams use `FastImage` or `expo-image` for aggressive disk caching.

38. Q: What's `Image.prefetch()`?

A: A static method that downloads and caches an image ahead of time (e.g., before navigating to a screen) so it renders instantly when the `Image` component mounts.

39. Q: How do you support multiple pixel densities for local images?

A: By providing `@2x` and `@3x` suffixed files alongside the base image (e.g., `logo.png`, `logo@2x.png`, `logo@3x.png`); RN's bundler auto-selects the right one for the device.

40. Q: What does `onError` on `Image` do?

A: It fires a callback with error info if the image fails to load (bad URL, network failure), letting you show a fallback UI.

41. Q: Can you apply `borderRadius` to make a circular avatar image?

A: Yes — setting equal `width`, `height`, and `borderRadius: width/2` produces a circular image, same technique as CSS.

42. Q: What's the difference between `Image` and `ImageBackground`?

A: `ImageBackground` renders an image as a background layer behind child components (like CSS `background-image`), whereas plain `Image` is a standalone element that can't contain children.

43. Q: Does `Image` support GIFs and WebP?

A: Yes on both platforms for basic display, though animated GIF support and performance vary; WebP requires enabling it in native build config on older RN/Android setups.

44. Q: How can loading many images in a list hurt performance?

A: Uncontrolled image loading (no caching, no downsizing, no lazy load) can spike memory and jank scrolling; solutions include list virtualization (`FlatList`), image resizing on the server, and caching libraries.

45. Q: What accessibility prop should images convey meaning have?

A: `accessibilityLabel`, which provides a text description read by screen readers, similar to `alt` text on the web.

Section D — Button (10)

46. Q: What platforms styling does the built-in `Button` support?

A: Very limited — mainly `color` (text/tint color) and `title`; you cannot control padding, font, border, or background shape beyond that.

47. Q: Why do most production apps avoid `Button`?

A: Because its styling API is too restrictive for custom designs; teams build custom pressable components instead for full control over appearance and interaction states.

48. Q: How do you disable a `Button`?

A: Set the `disabled={true}` prop, which also visually dims it and blocks `onPress` from firing.

49. Q: What prop handles a tap on `Button`?

A: `onPress`, a required prop taking a callback function.

50. Q: Does `Button` look the same on iOS and Android?

A: No — it renders with platform-native styling by default (different shapes/colors), reinforcing platform-appropriate UI conventions out of the box.

51. Q: Can `Button` show a loading state natively?

A: No, there's no built-in loading prop; you'd conditionally render an `ActivityIndicator` in place of the button or build a custom component.

52. Q: What's the `accessibilityLabel` default for `Button`?

A: It defaults to the `title` text if no explicit `accessibilityLabel` is provided.

53. Q: Is `Button` good for learning React Native fundamentals?

A: Yes — it's a simple way to learn `onPress` handling, `disabled` states, and prop-driven UI before moving to more flexible touchable components.

54. Q: Can you nest custom children inside `Button`?

A: No — `Button` only accepts a `title` string prop, not arbitrary children, unlike `Pressable` or `TouchableOpacity`.

55. Q: What color prop controls Android `Button` background?

A: `color` sets the button's background color on Android, but only the text color on iOS — a platform inconsistency worth knowing for interviews.

Section E — TextInput (20)

56. Q: What's the primary event for capturing typed text?

A: `onChangeText`, which fires with the current string value on every keystroke; there's also `onChange` which gives a full native event object.

57. Q: How do you show a numeric keyboard?

A: Set `keyboardType="numeric"` (or `"number-pad"`, `"decimal-pad"`, `"phone-pad"` depending on desired input set).

58. Q: How do you make a password field?

A: Set `secureTextEntry={true}`, which masks input characters.

59. Q: What does `autoCapitalize` control?

A: How the OS auto-capitalizes typed text: `'none'`, `'sentences'`, `'words'`, or `'characters'`.

60. Q: How do you detect the "submit"/"return" key press?

A: Via `onSubmitEditing`, fired when the user presses the return/done key on the keyboard.

61. Q: How do you programmatically focus a `TextInput`?

A: Get a ref to it and call `ref.current.focus()`; `.blur()` dismisses focus similarly.

62. Q: What's the difference between controlled and uncontrolled `TextInput` usage?

A: Controlled means the `value` prop is driven by React state updated via `onChangeText`; uncontrolled means you let the native input manage its own text and only read it via ref or `defaultValue`.

63. Q: How do you limit input length?

A: `maxLength` prop caps the number of characters accepted.

64. Q: What prop hides the return key's default label and customizes it?

A: `returnKeyType` (e.g., `'done'`, `'go'`, `'search'`, `'next'`) changes the label/icon on the keyboard's action key.

65. Q: How do you make a multiline text input (like a comment box)?

A: Set `multiline={true}`, and optionally control height via `numberOfLines` (Android) or dynamic `onContentSizeChange`.

66. Q: What's `autoCorrect` for?

A: A boolean prop to enable/disable the OS's autocorrect suggestions while typing.

67. Q: How do you detect focus/blur events?

A: Via the `onFocus` and `onBlur` callback props.

68. Q: What does `editable={false}` do?

A: Makes the input read-only/non-interactive without visually looking fully disabled like a grayed out field (styling for disabled look is manual).

69. Q: How do you set a placeholder and its color?

A: `placeholder` for the text, `placeholderTextColor` for its color, since regular text `color` style doesn't affect placeholder text.

70. Q: How does `TextInput` interact with the keyboard covering the input?

A: RN doesn't auto-adjust the layout; you typically wrap the screen in `KeyboardAvoidingView` or use a library like `react-native-keyboard-aware-scroll-view` to shift content up.

71. Q: What's `clearButtonMode` (iOS only)?

A: Shows/hides a native "clear text" (X) button inside the input, controlled by values like `'never'`, `'while-editing'`, `'always'`.

72. Q: How do you validate email input at the keyboard level?

A: Set `keyboardType="email-address"` to surface `@` and `.com` shortcuts, though actual validation logic must still be done in JS.

73. Q: What ref method clears the text field?

A: `ref.current.clear()` clears the input's text natively.

74. Q: Can `TextInput` support rich formatted text (bold/italic mixed)?

A: Not natively — RN's `TextInput` only handles plain text editing; rich text editors require third-party native modules or custom implementations.

75. Q: Why is debouncing `onChangeText` often necessary?

A: Because it fires on every keystroke, so expensive operations like API calls (search-as-you-type) should be debounced/throttled to avoid excessive network requests and re-renders.

Section F — ScrollView (15)

76. Q: Why is `ScrollView` risky for long lists?

A: It renders all children eagerly regardless of visibility, so a long or dynamic list can consume excessive memory and cause slow initial render/lag.

77. Q: When is `ScrollView` an appropriate choice?

A: For short, fixed-length content, like a settings screen or a form, where the full content is known and small enough to render upfront.

78. Q: How do you make `ScrollView` scroll horizontally?

A: Set `horizontal={true}`.

79. Q: What's `contentContainerStyle` for?

A: It styles the inner scrollable content container (e.g. padding, alignment), as distinct from `style`, which styles the outer scroll wrapper itself.

80. Q: How do you hide the scroll indicator?

A: `showsVerticalScrollIndicator={false}` or `showsHorizontalScrollIndicator={false}` depending on orientation.

81. Q: What is `pagingEnabled` used for?

A: Snaps scrolling to full-page/view increments, commonly used for horizontal carousels or onboarding screens.

82. Q: How do you detect scroll position changes?

A: The `onScroll` event callback, often combined with `scrollEventThrottle` to control how frequently it fires.

83. Q: What does `scrollEventThrottle` control?

A: How often (in milliseconds) the `onScroll` event fires during scrolling; a low value gives smoother tracking at higher performance cost.

84. Q: How do you programmatically scroll to a position?

A: Using a ref and calling `scrollTo({ x, y, animated })` or `scrollToEnd({ animated })`.

85. Q: Can `ScrollView` nest another `ScrollView` with the same scroll direction?

A: It's discouraged and can cause gesture conflicts; nesting is generally only safe with different scroll directions (e.g., vertical outer, horizontal inner) or by disabling scroll on one.

86. Q: What's `keyboardShouldPersistTaps` for?

A: Controls whether taps on child components (like buttons) dismiss the keyboard first or register normally, useful when a `TextInput` and tappable results list coexist.

87. Q: Does `ScrollView` support pull-to-refresh?

A: Yes, via the `refreshControl` prop, typically passed a `<RefreshControl>` element wired to `refreshing` state and `onRefresh` callback.

88. Q: How do you disable bounce/overscroll effects on iOS?

A: Set `bounces={false}`.

89. Q: What's the performance impact of `removeClippedSubviews` on `ScrollView`?

A: It can improve memory usage by detaching offscreen views from the native hierarchy, but it's an older optimization more reliably handled by `FlatList`'s virtualization today.

90. Q: Can `ScrollView` be combined with `Animated` for parallax/header effects?

A: Yes — commonly `Animated.ScrollView` (or `Animated.createAnimatedComponent(ScrollView)`) drives an animated value from `onScroll` for header collapse/parallax effects.

Section G — FlatList (25)

91. Q: What problem does `FlatList` solve compared to `ScrollView`?

A: It virtualizes rendering — only items currently visible (plus a small buffer) are mounted, dramatically reducing memory and improving performance for long or dynamic lists.

92. Q: What are the two required props for `FlatList`?

A: `data` (the array of items) and `renderItem` (a function returning the JSX for each item).

93. Q: What is `keyExtractor` for?

A: It returns a unique string key per item, used by React for efficient reconciliation instead of relying on array index by default.

94. Q: Why is using array index as a key discouraged in `FlatList`?

A: Because if items are inserted, removed, or reordered, index-based keys cause React to misattribute state/identity to the wrong items, leading to bugs and unnecessary re-renders.

95. Q: What does `initialNumToRender` control?

A: How many items are rendered in the very first batch/pass, balancing time-to-first-paint against on-screen completeness.

96. Q: What's `windowSize` in `FlatList`?

A: A multiplier (default 21) of the viewport size that determines how much content above/below the visible area stays rendered, trading memory for scroll smoothness.

97. Q: How do you implement infinite scroll/pagination with `FlatList`?

A: Use `onEndReached` (fired when scroll nears the list end) combined with `onEndReachedThreshold` to trigger fetching the next page of data.

98. Q: What's `ListEmptyComponent`?

A: A component rendered when `data` is an empty array, commonly used for "no results" states.

99. Q: What's `ListHeaderComponent` / `ListFooterComponent`?

A: Components rendered once at the very top/bottom of the list content, outside the repeating item template — useful for titles, loaders, or "load more" indicators.

100. Q: How do you render a grid instead of a single column?

A: Set `numColumns` greater than 1; items are automatically wrapped into rows.

101. Q: What is `getItemLayout` used for?

A: It lets you pre-compute each item's `length`, `offset`, and `index` when item sizes are fixed/known, skipping dynamic measurement and enabling instant `scrollToIndex`.

102. Q: How do you scroll to a specific item programmatically?

A: Using a ref and calling `scrollToIndex({ index, animated })` or `scrollToItem`.

103. Q: What causes items to "flash" or remount unexpectedly in `FlatList`?

A: Poor `keyExtractor` implementation or new inline function/object props on every render (e.g. inline `renderItem` closures) causing unnecessary re-renders of `React.memo`-wrapped item components.

104. Q: How can you optimize `renderItem` performance?

A: Memoize the item component with `React.memo`, use a stable `keyExtractor`, avoid creating new inline functions/objects per render, and use `useCallback` for `renderItem` itself.

105. Q: What's `refreshControl` in `FlatList` used for?

A: Same as in `ScrollView` — enables pull-to-refresh behavior via `onRefresh` and `refreshing` props.

106. Q: What does `removeClippedSubviews` do in `FlatList`?

A: On Android particularly, it detaches views that are scrolled offscreen from the native view hierarchy to reduce memory, though it can sometimes cause rendering glitches if misused.

107. Q: How is `FlatList` different from `SectionList`?

A: `SectionList` renders grouped data with section headers (like a contacts app grouped by letter), accepting a `sections` array of `{ title, data }` objects instead of a flat `data` array.

108. Q: What's `onViewableItemsChanged` used for?

A: A callback that reports which items are currently considered "viewable" on screen, useful for tracking impressions/analytics or triggering video autoplay.

109. Q: What's the underlying architecture behind `FlatList`?

A: It's built on top of `VirtualizedList`, which itself handles windowing/virtualization logic; `FlatList` is a more ergonomic, opinionated wrapper around it.

110. Q: How do you add spacing between items?

A: Via `ItemSeparatorComponent`, which renders a component between (not around) each item, or by applying margin in the item's own style.

111. Q: Can `FlatList` handle horizontal scrolling?

A: Yes, via `horizontal={true}`, commonly used for carousels.

112. Q: What happens if `data` changes but item content changes without the array reference changing?

A: `FlatList` may not detect the update; use `extraData` prop to signal it should re-render when some external state (not part of `data` itself) changes.

113. Q: How do you debounce/throttle `onEndReached` firing multiple times?

A: By tracking an `isFetchingMore` flag in state/ref and short-circuiting duplicate calls until the previous fetch resolves.

114. Q: What's the tradeoff of a very large `windowSize`?

A: More off-screen content stays mounted, which smooths fast scrolling but increases memory usage and can hurt performance on lower-end devices.

115. Q: How do you persist scroll position when navigating away and back?

A: Manually track the scroll offset (via `onScroll`) and call `scrollToOffset` on remount, since `FlatList` does not automatically preserve scroll position across full unmounts.

Section H — SafeAreaView / StatusBar (10)

116. Q: What problem does `SafeAreaView` solve?

A: It automatically adds padding to avoid system UI intrusions — the iPhone notch/Dynamic Island, home indicator, and similar safe-area-exclusion zones — so content doesn't render underneath them.

117. Q: Does `SafeAreaView` work well on Android?

A: Historically limited/inconsistent; most teams use the community `react-native-safe-area-context` library, which provides more reliable, configurable insets across both platforms.

118. Q: What does `StatusBar` control?

A: The appearance (color, style, visibility, translucency) of the device's status bar at the top of the screen.

119. Q: How do you set light text on a dark status bar?

A: `<StatusBar barStyle="light-content" />` (iOS-oriented prop name, also respected on Android).

120. Q: How do you hide the status bar entirely?

A: `<StatusBar hidden={true} />`.

121. Q: Can `StatusBar` settings differ per screen in a navigator?

A: Yes — placing a `<StatusBar>` component in each screen lets it override global settings when that screen is focused, common in navigation-heavy apps.

122. Q: What's `translucent` on Android `StatusBar`?

A: Makes the status bar transparent, letting your content render behind it, requiring you to manually add top padding/insets to avoid overlap.

123. Q: What's the `useSafeAreaInsets` hook for?

A: From `react-native-safe-area-context`, it returns numeric inset values (`top`, `bottom`, `left`, `right`) so you can apply padding/margin manually instead of using `SafeAreaView`'s wrapper behavior.

124. Q: Why might you avoid `SafeAreaView` in favor of manual insets?

A: For finer control — e.g., applying safe-area padding only to specific edges, or when a component (like a bottom tab bar) needs custom inset-aware styling rather than blanket padding.

125. Q: Does `SafeAreaView` affect landscape orientation?

A: Yes, insets adjust dynamically based on orientation (e.g., a notch appearing on the side in landscape), which is why libraries recalculate insets on rotation.

Section I — Other Core/Common Components (25)

126. Q: What's `Pressable` and why was it introduced?

A: A modern, highly configurable touch-handling component that unifies and extends the older Touchable* API, exposing granular states (`pressed`) and fine-tuned hit-slop/press-delay configuration.

127. Q: What's the difference between `TouchableOpacity` and `TouchableHighlight`?

A: `TouchableOpacity` reduces opacity on press for visual feedback, while `TouchableHighlight` overlays/darkens a background color underlay on press — both are largely superseded by `Pressable`.

128. Q: What does `TouchableWithoutFeedback` do?

A: Registers touches without any built-in visual feedback, often used to dismiss a keyboard when tapping outside an input.

129. Q: How does `Pressable`'s `style` prop support dynamic styling?

A: `style` can accept a function `({ pressed }) => ({...})`, letting you conditionally style based on interaction state without manual `useState`.

130. Q: What's `hitSlop` used for?

A: It expands the touchable area beyond a component's visual bounds (e.g., `{ top: 10, bottom: 10, left: 10, right: 10 }`), useful for small icons needing a larger tap target.

131. Q: What's `Modal` used for?

A: Rendering content above the rest of the app in a separate native layer, commonly for dialogs, full-screen overlays, or action sheets.

132. Q: What are `Modal`'s `animationType` options?

A: `'none'`, `'slide'`, and `'fade'`, controlling how the modal transitions in/out.

133. Q: What's `Modal`'s `presentationStyle` (iOS)?

A: Controls how the modal visually presents — `'fullScreen'`, `'pageSheet'`, `'formSheet'`, `'overFullScreen'` — mapping to native iOS presentation styles.

134. Q: How do you handle the Android hardware back button with `Modal`?

A: Via the `onRequestClose` prop, required on Android, which fires when the back button is pressed while the modal is open.

135. Q: What's `ActivityIndicator`?

A: A native platform-styled loading spinner component, configurable via `size` (`'small'`/`'large'`) and `color`.

136. Q: What's `Switch`?

A: A native toggle/checkbox-style boolean input, styled per-platform, controlled via `value` and `onValueChange`.

137. Q: What does `KeyboardAvoidingView` do?

A: Automatically adjusts its height, position, or bottom padding when the keyboard appears, preventing inputs from being obscured; behavior is controlled via the `behavior` prop (`'padding'`, `'height'`, `'position'`).

138. Q: Why does `KeyboardAvoidingView` behave differently per platform?

A: Because iOS and Android handle keyboard-triggered resizing differently at the OS level, so the recommended `behavior` value often differs (`'padding'` on iOS, sometimes unnecessary or `'height'` on Android).

139. Q: What is `SectionList` best suited for?

A: Grouped, categorized data — like an alphabetically sectioned contact list — rendering `renderSectionHeader` per group alongside virtualized items.

140. Q: What's `VirtualizedList`?

A: The lower-level abstraction underlying `FlatList` and `SectionList`, exposing more granular virtualization control for building custom high-performance lists.

141. Q: What's `Alert`?

A: An API (not a visual component you place in JSX) for showing native OS alert dialogs via `Alert.alert(title, message, buttons)`.

142. Q: What's the `Linking` API used for alongside components?

A: Opening external URLs, deep links, or other apps (e.g., `Linking.openURL('https://...')`), often triggered from a `Pressable`/`Text` `onPress`.

143. Q: What's `RefreshControl` and where is it used?

A: A component passed to `ScrollView`/`FlatList`'s `refreshControl` prop to implement native pull-to-refresh with a spinner tied to `refreshing` state.

144. Q: What's `FlatList`'s relationship to `SectionList` in terms of code reuse?

A: Both share the same `VirtualizedList` core, so performance techniques (memoization, `keyExtractor`, `getItemLayout`) generally transfer between them.

145. Q: What is `Dimensions`/layout-related but not a visual component — why mention it here?

A: It's an API (not JSX component) frequently used alongside core components for responsive sizing — see Topic 2 for full coverage.

146. Q: What's the purpose of `TouchableNativeFeedback` (Android-only)?

A: Provides Android's native ripple effect on press, giving material-design-consistent feedback; it has no iOS equivalent and is being folded into `Pressable`'s `android_ripple` config.

147. Q: How does `Pressable` replace `TouchableNativeFeedback`'s ripple?

A: Via the `android_ripple` prop, letting you configure ripple color/radius/borderless behavior directly without a separate component.

148. Q: What's `InputAccessoryView` (iOS)?

A: A component that renders a custom view pinned above the keyboard (like a "Done" toolbar), useful for custom input accessory bars.

149. Q: Why is component composition (View + Text + Pressable) usually preferred over monolithic built-ins like `Button`?

A: It offers full control over styling, layout, animation, and accessibility, matching design systems precisely rather than being constrained by a native component's limited prop surface.

150. Q: What's a common interview trap question about `View` and `Text` nesting?

A: Asking why `<View>Hello</View>` throws an error — testing whether the candidate understands RN's strict text-in-`Text`-only rule, unlike HTML's permissive text nodes anywhere.

React Native Interview Questions — Topic 2: Styling & Layout (150 Q&A)

Section A — StyleSheet.create (15)

1. Q: What does `StyleSheet.create` actually do?

A: It takes a plain object of style definitions and returns an object mapping each key to an internal numeric ID (in the old architecture) or an optimized reference, avoiding creating a brand-new style object on every render.

2. Q: Is `StyleSheet.create` required to style components?

A: No — you can pass plain JS objects directly to `style`, but `StyleSheet.create` is recommended for performance and tooling benefits.

3. Q: What performance benefit does `StyleSheet.create` give?

A: It lets styles be sent to the native side once and referenced by ID afterward, rather than being serialized and re-sent as fresh objects on every render.

4. Q: Does `StyleSheet.create` validate style properties?

A: Yes, to an extent — invalid or misspelled style keys often produce clearer warnings/errors than plain objects would, aiding debugging.

5. Q: Can you combine multiple styles on one component?

A: Yes, by passing an array: `style={[styles.base, styles.active]}` — later styles in the array override earlier ones for conflicting keys.

6. Q: How do you conditionally apply a style?

A: Common pattern: `style={[styles.button, isActive && styles.buttonActive]}`, relying on `false`/`null` being ignored in the style array.

7. Q: What's `StyleSheet.flatten()` used for?

A: It merges an array of styles (including IDs and objects) into a single plain object, useful when you need to inspect or manipulate computed style values in JS.

8. Q: What's `StyleSheet.absoluteFill`?

A: A shorthand style object equivalent to `{ position: 'absolute', top: 0, left: 0, right: 0, bottom: 0 }`, commonly used for full-bleed overlays.

9. Q: What's `StyleSheet.hairlineWidth`?

A: A cross-platform constant representing the thinnest line renderable on the current device's screen density, used for crisp 1px-look borders/dividers.

10. Q: Can `StyleSheet.create` styles be dynamic/computed at runtime?

A: Not directly inside `create` itself efficiently for per-render values — dynamic values (like from state) are usually passed as separate inline style objects merged with static `StyleSheet.create` styles.

11. Q: Why is defining styles outside the component function/render method recommended?

A: To avoid recreating style objects on every render, which would defeat memoization benefits and could trigger unnecessary re-renders in memoized children if styles are passed as props.

12. Q: Does `StyleSheet` support media-query-like conditional styles natively?

A: No — there's no native `@media` equivalent; responsive styling must be done in JS with `Dimensions`/`useWindowDimensions` and conditional logic.

13. Q: What TypeScript types are commonly used with `StyleSheet.create`?

A: `ViewStyle`, `TextStyle`, and `ImageStyle` from `react-native`, often combined into a typed style object via `StyleSheet.create<{...}>({...})` for compile-time safety.

14. Q: Can you spread one `StyleSheet` style into another plain object?

A: Yes, since flattened/created styles behave like accessible objects/IDs; spreading is possible but often unnecessary — arrays are the idiomatic composition method.

15. Q: What happens if two styles in an array define the same property?

A: The later style in the array wins — array order determines precedence, just like CSS cascade but simpler (no specificity rules, just array order).

Section B — Flexbox Fundamentals (25)

16. Q: What layout engine powers Flexbox in React Native?

A: Yoga, an open-source, cross-platform layout engine written in C++ that implements a Flexbox-like specification consistently across iOS, Android, and other platforms.

17. Q: Why is there no CSS Grid in RN?

A: Because Yoga implements only Flexbox, not the full CSS layout spec; grid-like layouts must be built manually using nested Flexbox rows/columns or via `numColumns` in `FlatList`.

18. Q: What is the default `flexDirection` in React Native?

A: `'column'` — the opposite of the web default (`'row'`), which is one of the most common gotchas for developers coming from web CSS.

19. Q: What does `flex: 1` mean on a component?

A: It tells the component to grow and take up all available remaining space along the main axis relative to its siblings' flex values.

20. Q: What's the difference between `flexGrow`, `flexShrink`, and `flexBasis`?

A: `flexGrow` controls how much a component expands into extra space, `flexShrink` controls how much it shrinks when space is tight, and `flexBasis` sets its initial size before growing/shrinking is applied.

21. Q: How does `flex: 1` differ from setting an explicit `height`/`width`?

A: `flex: 1` is relative/proportional and adapts to available space and sibling flex values, while explicit dimensions are fixed regardless of container size or siblings.

22. Q: What's the "main axis" vs "cross axis"?

A: The main axis follows `flexDirection` (e.g., vertical for `'column'`); the cross axis is perpendicular to it — `justifyContent` operates on the main axis, `alignItems` on the cross axis.

23. Q: What does `justifyContent` control?

A: Alignment/distribution of children along the main axis: `'flex-start'`, `'flex-end'`, `'center'`, `'space-between'`, `'space-around'`, `'space-evenly'`.

24. Q: What does `alignItems` control?

A: Alignment of children along the cross axis: `'flex-start'`, `'flex-end'`, `'center'`, `'stretch'`, `'baseline'`.

25. Q: What's the difference between `alignItems` and `alignSelf`?

A: `alignItems` is set on the parent and applies to all children by default; `alignSelf` is set on an individual child and overrides the parent's `alignItems` just for that child.

26. Q: What's `alignContent` for, and how does it differ from `alignItems`?

A: `alignContent` controls spacing between *multiple lines* of wrapped content (only relevant with `flexWrap: 'wrap'` and multiple rows/columns), whereas `alignItems` aligns items within a single line.

27. Q: What does `flexWrap: 'wrap'` do?

A: Allows children to wrap onto multiple lines/rows instead of being forced to shrink or overflow along a single line.

28. Q: If `flexDirection` is `'row'`, which axis does `justifyContent` control?

A: The horizontal axis (main axis for row layouts) — this is a very common interview trick question testing axis comprehension.

29. Q: What's the default value of `alignItems`?

A: `'stretch'` — children stretch to fill the cross axis unless a fixed dimension along that axis is specified.

30. Q: What happens if a child has no explicit `height` in a `column` flex container with `alignItems: 'stretch'`?

A: Its height is determined by its content (or flex-grow behavior along the main axis), while `stretch` affects the cross axis (width, in a column container).

31. Q: How do you center a single child both horizontally and vertically?

A: Set the parent's `flex: 1`, `justifyContent: 'center'`, and `alignItems: 'center'`.

32. Q: What's `flexDirection: 'row-reverse'`?

A: Lays out children horizontally but in reverse order (last child appears first).

33. Q: Can `flex` values be fractional or unequal between siblings?

A: Yes — siblings with `flex: 1` and `flex: 2` split remaining space proportionally, with the second taking twice as much as the first.

34. Q: What's a common bug when mixing `flex: 1` with `ScrollView`?

A: Applying `flex: 1` directly to `ScrollView`'s `style` without `flexGrow: 1` on `contentContainerStyle` can cause content to not fill available space or the ScrollView to not size correctly.

35. Q: How does `flexBasis: 'auto'` behave?

A: The item's size is based on its own content/explicit width-height before any growing or shrinking is applied — the default behavior when `flexBasis` isn't set.

36. Q: What's the effect of `flexShrink: 0`?

A: Prevents the item from shrinking below its base size even if the container doesn't have enough space, which can cause overflow.

37. Q: How do you create equal-width columns with Flexbox?

A: Give each column `flex: 1` inside a `flexDirection: 'row'` parent; they'll split the available width equally.

38. Q: What's the interview-relevant difference between RN Flexbox and web Flexbox besides direction default?

A: RN doesn't support all CSS flex shorthand syntax (`flex: 1 1 auto` isn't a single string prop — you set `flexGrow`/`flexShrink`/`flexBasis` individually or use the simplified numeric `flex` prop), and percentage support/behavior can differ slightly.

39. Q: Does RN support `gap` for spacing between flex children?

A: Yes in modern RN versions (`gap`, `rowGap`, `columnGap` style props are supported), removing the need for manual margin-based spacing hacks.

40. Q: What's a common technique for spacing children before `gap` was supported?

A: Applying margin to all but the first/last child, often via a custom `Spacer` component or conditional margin logic based on index in a mapped list.

Section C — flexDirection / justifyContent / alignItems / alignSelf / flexWrap Deep Dive (20)

41. Q: How would you build a horizontal navbar with items spread to the edges?

A: `flexDirection: 'row'` with `justifyContent: 'space-between'` on the container.

42. Q: How do you vertically center text next to an icon in a row?

A: `flexDirection: 'row'` with `alignItems: 'center'` on the parent.

43. Q: What's the effect of `justifyContent: 'space-evenly'` vs `'space-between'`?

A: `'space-between'` places equal space only *between* items (none at the edges), while `'space-evenly'` distributes equal space between items *and* at both edges.

44. Q: How do you make one child stick to the bottom of a `flex: 1` column container?

A: `justifyContent: 'flex-end'` on the parent, or give preceding siblings `flex: 1` to push the target child down, or use `marginTop: 'auto'` on the target child (RN supports auto margins).

45. Q: Does React Native support `margin: 'auto'` for pushing elements apart?

A: Yes, RN supports `marginLeft: 'auto'`, `marginTop: 'auto'`, etc., which behave like CSS auto margins in flex containers, useful for pushing a single item to one side.

46. Q: How do you align one item differently from its row siblings?

A: Apply `alignSelf` on that specific item, overriding the parent's `alignItems` value just for it.

47. Q: What's the effect of combining `flexWrap: 'wrap'` with `flexDirection: 'row'`?

A: Items lay out left-to-right and wrap to a new "row line" when they run out of horizontal space, useful for tag clouds or grid-like layouts.

48. Q: How would you build a responsive grid of cards without `FlatList`'s `numColumns`?

A: A `flexDirection: 'row'`, `flexWrap: 'wrap'` container with each card given a fixed or percentage-based width (e.g., `width: '48%'`) so a fixed number fit per line.

49. Q: What's the layout gotcha with `alignItems: 'stretch'` and `Text`?

A: Text components don't stretch to fill cross-axis width in the same intuitive way block elements do on the web; text width is generally driven by content unless explicitly constrained.

50. Q: How does `baseline` alignment work in `alignItems`?

A: It aligns children so their text baselines line up horizontally, useful when mixing different font sizes in a row (e.g., a price with a large number and small currency label).

51. Q: What happens when you set both `flexGrow: 1` and an explicit `width` on the same element?

A: The explicit `width` generally acts as the starting basis, but `flexGrow` can still cause the element to expand beyond it if there's available space, depending on `flexBasis` interplay.

52. Q: How would you build a two-column layout where one column is fixed-width and the other flexible?

A: `flexDirection: 'row'` parent; fixed column gets an explicit `width` (e.g., `width: 80`), flexible column gets `flex: 1` to consume remaining space.

53. Q: What's the risk of deeply nesting flex containers with `flex: 1`?

A: Each level adds layout calculation overhead, and improperly balanced `flex` values across nested levels can cause components to collapse to zero height/width unexpectedly.

54. Q: Why might a `flex: 1` child render with zero height inside a `ScrollView`?

A: Because `ScrollView`'s content container doesn't have a bounded height by default (it grows to fit content), so `flex: 1` has no fixed parent size to be relative to, collapsing the child.

55. Q: How do you fix that zero-height issue?

A: Set `flexGrow: 1` on the `ScrollView`'s `contentContainerStyle` (not `flex: 1`) so the content area is allowed to grow to fill the viewport while still being scrollable if content exceeds it.

56. Q: What's `flexDirection: 'column-reverse'` used for?

A: Stacks children vertically but in reverse order (last child on top) — useful for chat interfaces where new messages should visually anchor at the bottom.

57. Q: How do you evenly space icons in a bottom tab bar?

A: `flexDirection: 'row'`, `justifyContent: 'space-around'` or `'space-between'`, each tab item often also getting `flex: 1` for equal-width tap targets.

58. Q: What's a common Flexbox debugging technique in RN (no browser devtools inspector by default)?

A: Temporarily adding contrasting `backgroundColor` to each nested `View` to visually reveal boundaries, or using React DevTools/Flipper's layout inspector.

59. Q: How does `alignSelf: 'stretch'` differ when a fixed `width` is also specified on the child?

A: The explicit `width` takes precedence — `alignSelf: 'stretch'` only has an effect when no conflicting fixed cross-axis dimension is set.

60. Q: Why might interviewers ask you to draw out a layout with only `flexDirection`, `justifyContent`, and `alignItems`?

A: Because mastering these three properties covers the vast majority of real-world RN layouts, and testing them reveals whether a candidate truly understands axis-relative reasoning versus memorized snippets.

Section D — Positioning (15)

61. Q: What are the two `position` values in RN?

A: `'relative'` (default) and `'absolute'` — RN does not support `'fixed'` or `'sticky'` like web CSS.

62. Q: How does `position: 'absolute'` behave relative to its parent?

A: It's positioned relative to its nearest ancestor (the parent `View`), using `top`, `left`, `right`, `bottom` offsets, and is removed from normal flex flow.

63. Q: How do you center an absolutely positioned overlay?

A: Combine `position: 'absolute'` with `top: 0, left: 0, right: 0, bottom: 0` (or `StyleSheet.absoluteFill`) plus `justifyContent: 'center'`, `alignItems: 'center'` on that same layer.

64. Q: What happens to sibling layout when one child is `position: 'absolute'`?

A: That child is taken out of the normal Flexbox flow entirely; it doesn't affect the size or position calculations of its siblings.

65. Q: How does `zIndex` behave in RN compared to web?

A: It works similarly (higher values render on top) but only among sibling views in the same stacking context — there's no true global stacking context management like advanced CSS.

66. Q: What's a common use case for `position: 'absolute'`?

A: Badges/notification dots on icons, floating action buttons, custom modals/tooltips, and overlay headers on top of scrollable content.

67. Q: Can `position: 'absolute'` elements overflow their parent bounds?

A: Yes, unless the parent has `overflow: 'hidden'` set, in which case they'll be clipped at the parent's edges.

68. Q: How do you create a badge in the top-right corner of an icon?

A: Wrap the icon in a `View` with `position: 'relative'`, then give the badge `position: 'absolute'`, `top: -4, right: -4` (or similar small offsets).

69. Q: Does RN support `position: 'sticky'` for headers?

A: Not as a style prop; sticky headers within a `ScrollView` are achieved via `stickyHeaderIndices` prop on `ScrollView`, or manually with animated positioning based on scroll offset.

70. Q: What's `stickyHeaderIndices`?

A: A `ScrollView` prop accepting an array of child indices that should remain pinned to the top while scrolling past them, mimicking sticky/fixed headers.

71. Q: How do negative margins interact with layout in RN?

A: They're supported and behave like CSS negative margins, pulling an element outside its normal box, often used for overlapping visual effects instead of absolute positioning in simple cases.

72. Q: What determines stacking order when `zIndex` is not set?

A: DOM/JSX order — later siblings render on top of earlier ones by default, same as painting order in most UI systems.

73. Q: Can you animate `zIndex`?

A: Technically yes via state changes, but since it's not a continuous/interpolatable value in the traditional `Animated` sense, it's usually toggled discretely rather than smoothly animated.

74. Q: What's the interaction between `position: 'absolute'` and percentage-based `width`?

A: Percentages resolve against the nearest positioned/sized ancestor's dimensions, same conceptual model as CSS, letting you build responsive absolutely-positioned overlays.

75. Q: Why might an absolutely positioned element not appear at all?

A: Common causes: the parent has no defined size (zero width/height) for percentages to resolve against, or the element is clipped by an ancestor's `overflow: 'hidden'`.

Section E — Dimensions & useWindowDimensions (15)

76. Q: What does `Dimensions.get('window')` return?

A: An object with `width`, `height`, `scale`, and `fontScale` for the app's usable window area (distinct from `'screen'`, which includes system UI in some cases).

77. Q: What's the difference between `Dimensions.get('window')` and `Dimensions.get('screen')`?

A: `'window'` reflects the visible application area; `'screen'` reflects the full physical screen dimensions, which can differ on platforms with split-screen or multi-window support.

78. Q: Why doesn't `Dimensions.get()` update automatically on rotation?

A: Because it's a synchronous snapshot taken at call time — it does not subscribe to changes, so a component relying only on it will show stale dimensions after rotation unless it manually re-queries.

79. Q: How do you react to dimension changes (e.g., rotation) with `Dimensions`?

A: Subscribe to the `'change'` event: `Dimensions.addEventListener('change', callback)`, then update local state and remember to remove the listener on unmount.

80. Q: What's `useWindowDimensions()` and why is it preferred today?

A: A React hook that returns live `width`/`height` and automatically triggers a re-render on orientation/window size changes, removing the need for manual event listener boilerplate.

81. Q: How would you conditionally render a two-column vs one-column layout based on screen width?

A: `const { width } = useWindowDimensions(); const columns = width > 600 ? 2 : 1;` then apply that to layout logic or `FlatList`'s `numColumns`.

82. Q: Is `useWindowDimensions` reactive to split-screen/multi-window resizing on tablets?

A: Yes — since it's hook-based and subscribes to native dimension change events, it re-renders when the app's window is resized, e.g., in Android split-screen mode.

83. Q: What's a percentage-based width alternative to using `Dimensions`?

A: Using string percentage values directly in styles (e.g., `width: '50%'`), which resolves relative to the parent's size without needing JS calculation at all.

84. Q: When would you still need `Dimensions`/`useWindowDimensions` over percentages?

A: When you need the actual pixel value for calculations (e.g., computing an image aspect-ratio height, positioning based on absolute pixel offsets, or comparing against breakpoints).

85. Q: What's a common breakpoint strategy pattern in RN (since there's no media query)?

A: Defining JS constants for breakpoints (e.g., `const isTablet = width >= 768`) and branching styles/layout conditionally based on those booleans.

86. Q: Does `Dimensions.get('window')` include the status bar height?

A: Generally yes, it reflects the usable app window, but exact behavior can vary by platform and is why insets (`SafeAreaView`/`useSafeAreaInsets`) are handled separately.

87. Q: How do you get notified before first render of the current dimensions without a flash of incorrect layout?

A: `useWindowDimensions()` provides the value synchronously on first render (no separate effect/listener needed like the older `Dimensions.get` + event pattern), avoiding a layout flash.

88. Q: Can `Dimensions` be used outside of a React component (e.g., in a utility file)?

A: Yes, `Dimensions.get('window')` is a plain static call usable anywhere, unlike hooks which are constrained to component/hook call rules.

89. Q: What's a caveat of hardcoding pixel values from `Dimensions.get()` at module load time?

A: The value is captured once at import time and won't update on rotation or window resize, silently causing stale layout — a classic RN interview gotcha.

90. Q: How does `useWindowDimensions` help with orientation-specific styling?

A: You can compare `width` vs `height` (or check both) to derive `isLandscape`, and conditionally swap `flexDirection` or layout structure accordingly, with automatic re-renders on rotation.

Section F — PixelRatio & Responsive Design (15)

91. Q: What is `PixelRatio` used for?

A: A utility for converting between React Native's density-independent pixels (dp) and actual physical device pixels, useful for pixel-perfect rendering like hairlines and crisp images.

92. Q: What does `PixelRatio.get()` return?

A: The device's pixel density ratio (e.g., 2 for `@2x` devices, 3 for `@3x` devices), similar to `window.devicePixelRatio` on the web.

93. Q: Why does pixel density matter for borders?

A: A `borderWidth: 1` in dp might render as 2 or 3 physical pixels on high-density screens, looking thicker than intended — `StyleSheet.hairlineWidth` compensates by using the thinnest renderable line.

94. Q: What's `PixelRatio.getFontScale()`?

A: Returns the font scaling factor based on the user's OS accessibility text-size settings, useful for calculations that need to account for enlarged text.

95. Q: What's `PixelRatio.roundToNearestPixel()` for?

A: Rounds a dp value to the nearest value that maps cleanly to a whole physical pixel on the current device, avoiding blurry sub-pixel rendering of borders/lines.

96. Q: How does `PixelRatio` relate to `@2x`/`@3x` image assets?

A: RN uses the device's pixel ratio to automatically select the appropriately suffixed local image asset, ensuring images look sharp without unnecessary over-fetching on lower-density devices.

97. Q: What's the difference between dp (density-independent pixels) and physical pixels?

A: Dp is a logical unit that RN layout uses consistently regardless of screen density; physical pixels are the actual hardware pixels, and the ratio between them varies per device.

98. Q: How would you create a "true 1px" border using `PixelRatio`?

A: `borderWidth: 1 / PixelRatio.get()`, which converts one physical pixel back into the equivalent dp value for the current device — though `StyleSheet.hairlineWidth` is the simpler built-in solution.

99. Q: What's a responsive font-scaling technique using `PixelRatio` or `Dimensions`?

A: Scaling font size relative to screen width (e.g., `fontSize: width * 0.05`) or using a normalization function that adjusts a base size against a reference device width.

100. Q: What's the risk of over-relying on pixel-based responsive scaling formulas?

A: It can produce inconsistent or extreme sizes on very small or very large/tablet screens if not clamped with min/max bounds, so most teams cap scaled values within a sensible range.

101. Q: How does `aspectRatio` style help with responsive images/containers?

A: It lets you specify a width-to-height ratio (e.g., `aspectRatio: 16/9`) so RN computes one dimension automatically from the other, avoiding manual pixel math for responsive media.

102. Q: Can `aspectRatio` be combined with `width: '100%'`?

A: Yes — a very common pattern: full-width container with `aspectRatio` set computes a proportional height automatically as the width changes across devices.

103. Q: What library is commonly used for scaling designs across device sizes (interview-relevant mention)?

A: Libraries like `react-native-size-matters` (`scale`, `verticalScale`, `moderateScale`) or custom scaling utilities based on a reference design width/height from Figma.

104. Q: Why is percentage-based sizing sometimes insufficient for responsiveness?

A: Because percentages only relate to the immediate parent, not the overall screen, so deeply nested percentage chains can produce unpredictable absolute sizes across devices.

105. Q: How do you handle safe, responsive spacing (margins/paddings) across device sizes?

A: Common approaches include a spacing scale (e.g., 4/8/16/24 multiples) applied consistently, or scaling functions tied to `useWindowDimensions`, rather than ad hoc fixed values everywhere.

Section G — Platform-Specific Styling (10)

106. Q: What's `Platform.OS` used for?

A: A string constant (`'ios'` or `'android'`) letting you branch logic/styles conditionally per platform.

107. Q: What's `Platform.select()`?

A: A utility that picks a value from an object keyed by platform (`ios`, `android`, `default`), cleanly consolidating platform-specific style/logic branches.

108. Q: How would you apply a different shadow implementation per platform?

A: Using `Platform.select({ ios: { shadowColor, shadowOffset, shadowOpacity, shadowRadius }, android: { elevation } })`, since shadows are implemented differently natively on each OS.

109. Q: What are platform-specific file extensions used for?

A: Files named `Component.ios.js` and `Component.android.js` let the bundler automatically pick the right implementation per platform without explicit `Platform.OS` checks in code.

110. Q: When would you use platform-specific files vs `Platform.select`?

A: Platform-specific files suit large behavioral/structural differences (different component trees entirely); `Platform.select` suits small, inline style/config differences within otherwise shared code.

111. Q: What's `Platform.Version`?

A: Returns the OS version number (e.g., Android API level, iOS version string), useful for conditionally applying fixes/features only needed on certain OS versions.

112. Q: Why might font rendering differ between iOS and Android even with identical `fontSize`?

A: Different default system fonts (San Francisco vs Roboto) have different metrics/line-heights, so identical numeric values can look visually different in size/spacing across platforms.

113. Q: How do you handle platform differences in default `TextInput` underline styling on Android?

A: Set `underlineColorAndroid: 'transparent'` (older RN) or ensure appropriate `borderBottomWidth`/`borderBottomColor` styling is applied consistently rather than relying on Android's default underline.

114. Q: What's a common interview question about `elevation` vs `shadow*` props?

A: Explaining that Android uses a single `elevation` numeric prop (leveraging Material Design's layered shadow system) while iOS requires separate `shadowColor`/`shadowOffset`/`shadowOpacity`/`shadowRadius` props — they are not interchangeable and must both be set for cross-platform parity.

115. Q: Does `Platform.select` support more than two platforms (e.g., web, macOS)?

A: Yes — with `react-native-web` or out-of-tree platforms like `react-native-macos`, `Platform.select` can include keys like `web`, `macos`, `windows` alongside `ios`/`android`/`default`.

Section H — Shadows / Borders / Elevation (10)

116. Q: What are the four iOS shadow style props?

A: `shadowColor`, `shadowOffset` (`{ width, height }`), `shadowOpacity`, and `shadowRadius`.

117. Q: Why doesn't `shadowOpacity` alone show a shadow on iOS?

A: Because all four shadow props typically need to be set together (color, offset, opacity, radius) for a visible, correctly styled shadow — omitting any one often makes it invisible or malformed.

118. Q: What does `elevation` do on Android?

A: Applies Material Design's elevation shadow system, where higher numeric values create a larger, more diffuse drop shadow beneath the component, also implicitly affecting stacking order.

119. Q: Can you customize shadow color on Android via `elevation`?

A: Historically no (elevation used a fixed system shadow color), though newer RN versions support `shadowColor` alongside `elevation` on Android for more control.

120. Q: What's the simplest border syntax for a full box border?

A: `borderWidth` + `borderColor` (with optional `borderRadius` for rounded corners), applied like standard CSS border shorthand concepts but as separate RN style props.

121. Q: How do you create a border on only one side (e.g., bottom divider)?

A: Use directional props like `borderBottomWidth` and `borderBottomColor` instead of the shorthand `borderWidth`/`borderColor`.

122. Q: Why might a `borderRadius` not visually clip a child `Image` inside a `View`?

A: Because `overflow: 'hidden'` must also be set on the parent `View`; `borderRadius` alone rounds the border but doesn't automatically clip children's content bounds.

123. Q: What's a cross-platform library often used to simplify shadow styling?

A: Libraries like `react-native-shadow-2` or custom `Platform.select` shadow utility functions that abstract the iOS/Android differences into a single reusable style prop.

124. Q: Can shadows be applied to `Text` directly?

A: Yes, via `textShadowColor`, `textShadowOffset`, and `textShadowRadius`, distinct from the `shadow*` box-shadow props used on `View`/`Image`.

125. Q: Why can shadows hurt performance if overused in long lists?

A: Shadow rendering (especially iOS's layer-based shadow rasterization) can be computationally expensive per view, so applying complex shadows to many list items can cause scroll jank.

Section I — Safe Area & Notches in Styling Context (10)

126. Q: How do safe area insets interact with custom headers?

A: Custom header components typically add `paddingTop: insets.top` (from `useSafeAreaInsets`) to avoid rendering under the status bar/notch, since a plain `View` has no automatic awareness.

127. Q: What's the risk of hardcoding a fixed status bar height value (e.g., `paddingTop: 20`)?

A: It breaks across devices with different notch/inset sizes (e.g., iPhone with Dynamic Island vs an older device), producing inconsistent spacing — dynamic insets should be used instead.

128. Q: How do bottom tab bars typically handle the home indicator safe area on iOS?

A: By adding `paddingBottom: insets.bottom` so tab bar content isn't obscured by or too close to the home indicator gesture area.

129. Q: Does `SafeAreaProvider` need to wrap the whole app?

A: Yes — `react-native-safe-area-context`'s `SafeAreaProvider` must wrap the app root so that `useSafeAreaInsets`/`SafeAreaView` throughout the tree can access correct inset values.

130. Q: How do insets change with landscape orientation on notched devices?

A: The notch may shift to the left or right safe-area edge instead of the top, so `insets.left`/`insets.right` become relevant in addition to `insets.top` for landscape layouts.

131. Q: What's a common bug when using `SafeAreaView` inside a nested component instead of at the screen root?

A: Applying it at the wrong level can cause double-padding (if nested inside another `SafeAreaView`) or insufficient padding (if applied too deep, missing the true screen edges).

132. Q: How do modals interact with safe areas?

A: Full-screen modals need their own safe-area handling since they render in a separate native layer outside the normal screen's `SafeAreaView` context.

133. Q: What's the edges prop on `SafeAreaView` (from `react-native-safe-area-context`) for?

A: It lets you specify which edges (`'top'`, `'bottom'`, `'left'`, `'right'`) should receive safe-area padding, useful when you only want inset padding on specific sides.

134. Q: How would you keep a floating action button above the home indicator?

A: Add `bottom: insets.bottom + baseOffset` to its absolute positioning style, combining safe-area insets with your own desired spacing.

135. Q: Why is testing on a real notched device (or accurate simulator) important for safe-area styling?

A: Because inset values and notch shapes vary significantly across devices, and incorrect assumptions can cause content clipping or excessive empty space that's easy to miss without device-accurate testing.

Section J — Units, Percentages, Aspect Ratio, Responsive Patterns (15)

136. Q: What unit system does RN use for most style values?

A: Unitless numbers representing density-independent pixels (dp/dip) — you don't write `px`, `em`, or `rem` as in CSS.

137. Q: Does RN support percentage strings for width/height?

A: Yes, e.g., `width: '50%'`, resolved relative to the parent container's corresponding dimension.

138. Q: Can percentages be used for `margin`/`padding`?

A: Support is more limited/inconsistent historically compared to `width`/`height`; explicit numeric values are generally more reliable for spacing.

139. Q: What's the difference between `aspectRatio` and manually computing height from width?

A: `aspectRatio` lets the native layout engine compute the missing dimension automatically and responsively, whereas manual computation requires JS logic (e.g., in `onLayout`) and doesn't update as fluidly with dynamic container resizing.

140. Q: How do you build a responsive card grid that adapts to tablet vs phone?

A: Combine `useWindowDimensions` to compute a dynamic `numColumns` for `FlatList`, or conditionally set flex-basis/width percentages based on breakpoint logic.

141. Q: What's `onLayout` used for in responsive design?

A: A callback fired after a component's layout is calculated, providing its actual rendered `x`, `y`, `width`, `height` — useful when you need real (not just requested) dimensions for further calculations.

142. Q: Why might `onLayout` fire multiple times?

A: It fires whenever the component's layout changes (e.g., due to content changes, orientation changes, or parent resizing), not just once on mount.

143. Q: What's a "responsive typography scale" pattern in RN?

A: Defining a small set of named font sizes (e.g., `xs`, `sm`, `md`, `lg`, `xl`) possibly scaled by screen width/`PixelRatio.getFontScale()`, ensuring consistent, accessible text sizing across the app.

144. Q: How do you avoid layout shift when an image is still loading?

A: Reserve space ahead of time using a fixed `aspectRatio` or known `width`/`height` so the surrounding layout doesn't jump once the image finishes loading.

145. Q: What's the tradeoff between fixed breakpoints and continuous scaling functions for responsiveness?

A: Fixed breakpoints (phone/tablet) are simpler to reason about and design for but can feel stepped/abrupt; continuous scaling functions feel smoother across sizes but are harder to visually predict/test for every value.

146. Q: How do orientation changes typically get handled in styling logic?

A: By deriving `isLandscape` from `useWindowDimensions` (comparing `width` vs `height`) and conditionally swapping `flexDirection` or dimension-dependent values in response.

147. Q: What's a common technique for consistent spacing systems across an app?

A: A shared spacing scale/theme object (e.g., `{ xs: 4, sm: 8, md: 16, lg: 24, xl: 32 }`) referenced throughout `StyleSheet.create` calls instead of arbitrary magic numbers.

148. Q: Why is `minWidth`/`maxWidth` useful in responsive RN layouts?

A: They constrain a flexible or percentage-based element so it doesn't become too small on tiny screens or too large/stretched on tablets, similar to their CSS counterparts.

149. Q: How would you explain RN's overall responsive design philosophy to an interviewer?

A: There's no built-in media-query system; responsiveness is achieved by combining Flexbox's relative sizing, `useWindowDimensions`/`PixelRatio` for JS-driven breakpoints, and consistent spacing/typography scales, all computed and re-rendered reactively rather than declared statically in CSS.

150. Q: What's a strong closing interview answer summarizing RN styling vs web CSS?

A: RN styling is CSS-inspired but intentionally reduced — no cascade/inheritance beyond `Text`, only Flexbox (no Grid/floats), unitless dp values instead of multiple CSS units, and JS-driven responsiveness instead of media queries — trading some familiar web ergonomics for a smaller, more predictable, cross-platform-consistent styling surface.